

1. Description de la méthode

a. Prototype

```
private System.Windows.Point LatLngToPoint(PointLatLng point)
```

b. Résumé

Cette méthode va projeter un point de coordonnées (lat, long) sur un plan 2D en utilisant la projection Mercator

c. Paramètres d'entrés

Paramètre d'entrée	Type	Description
point	Point	point à convertir

d. Valeur de retour

Cette méthode retourne le point représentant la projection du point passé en paramètre sur un plan 2D

2. Projection Mercator

La méthode LatLngToPoint comme expliqué dans le résumé présent dans la partie 1 va projeter les points de coordonnées (lat, long) sur un carte plane (plan 2D) à l'aide de la projection de Mercator

La projection Mercator s'applique en utilisant ce système d'équation :

$$x = n(\lambda - \lambda_0)$$

$$y = n \cdot \ln(\tan(\frac{\pi}{4} \cdot \frac{\varphi}{2}))$$

Légende :

Variable	Unité	Description
n	/	Facteur mettant à l'échelle de la carte plane le point calculé à l'aide de la formule
λ	radiant	Longitude du point
λ_0	radiant	Longitude du centre de la carte ou l'on va testé
φ	radiant	Latitude du point

Dans cette méthode ma carte sur laquelle je vais projeter mes points est basée sur le point de coordonnées GPS (0, 0) et a la même taille que la carte implémenté dans l'ihm. Cela me permettra de faciliter les calculs car $\lambda_0 = 0$

en prenant en compte ces informations on en vient à ce système d'équation :

$$x = w \cdot \frac{\lambda \cdot \pi}{2\pi}$$

$$y = \frac{h}{2} \cdot \frac{w}{2\pi} \cdot \ln(\tan(\frac{\pi}{4} \cdot \frac{\varphi}{2}))$$

Documentation Méthode LatLngToPoint

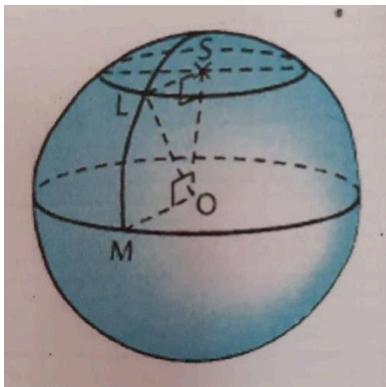
Légende :

Variable	Unité	Description
h	pixel	hauteur de la carte
w	pixel	largeur de la carte

ou n ici a été remplacé par $w \cdot \frac{\pi}{2\pi}$ pour x et par $\frac{h}{2} - \frac{w}{2\pi}$ pour y . ce remplacement sert à mettre à l'échelle le point que l'on veut projeter sur notre carte plane je m'explique :

Comme on l'a vu précédemment w représente la largeur de la carte si on souhaite intégrer toutes les longitude possible de notre globe terrestre sur la carte alors $w = 360^\circ/2\pi$ de ce fait $\frac{w}{2\pi} = 1 \text{ rad}$

Donc pour trouver l'équivalent de la position sur notre carte à sa position sur le globe terrestre il nous suffit de faire : $\lambda \cdot \frac{w}{2\pi}$ ce qui va mettre à l'échelle notre longitude par rapport à la carte. Cela marche car la terre est une sphère et que une longueur est souvent représenté à l'aide de l'angle comparé au centre de la sphère comme ceci :



sauf qu'il y a un problème car comme vous le savez peut être l'intervalle d'angle défini pour les longitudes par rapport au centre de notre globe terrestre est compris entre -180° et 180° ce qui nous donnerait des valeurs négative de pixel si on utilisait la formule telle qu'elle or ce n'est pas possible

donc pour régler cela on décale l'intervalle de $180^\circ/\pi$ afin d'éviter ces nombres négatifs d'où le $180/\pi$ ajouté à λ dans la formule

Pour les y c'est le même raisonnement mais cette fois ci c'est $\frac{h}{2}$ qui garantira que les valeurs restent sur la carte.

Si on prend par exemple l'équateur qui est de latitude 0 dans ce cas cela nous donne dans la formule : $\frac{h}{2} - 0$

Documentation Méthode LatLngToPoint

donc il ne nous restera que $\frac{h}{2}$ et l'équateur sera placé au milieu de la carte. En résumé grâce à ce $\frac{h}{2}$ les valeurs tourneront autour de l'équateur car :

$0 > \varphi$	$0 > \ln(\tan(\frac{\pi}{4} \cdot \frac{\varphi}{2}))$	valeurs décroissantes
$0 < \varphi$	$0 < \ln(\tan(\frac{\pi}{4} \cdot \frac{\varphi}{2}))$	valeurs croissantes

Etant donné que cette méthode n'est pas très longue on a qu'une seule partie qui implémente tout ce que j'ai expliqué plus tôt :

```
System.Windows.Point local_point = new System.Windows.Point();
double map_height = ihm.ActualHeight;
double map_width = ihm.ActualWidth;

//Conversion de la latitude et de la longitude en radian
double lat_rad = point.Lat * Math.PI / 180;
double long_rad = point.Lng * Math.PI / 180;

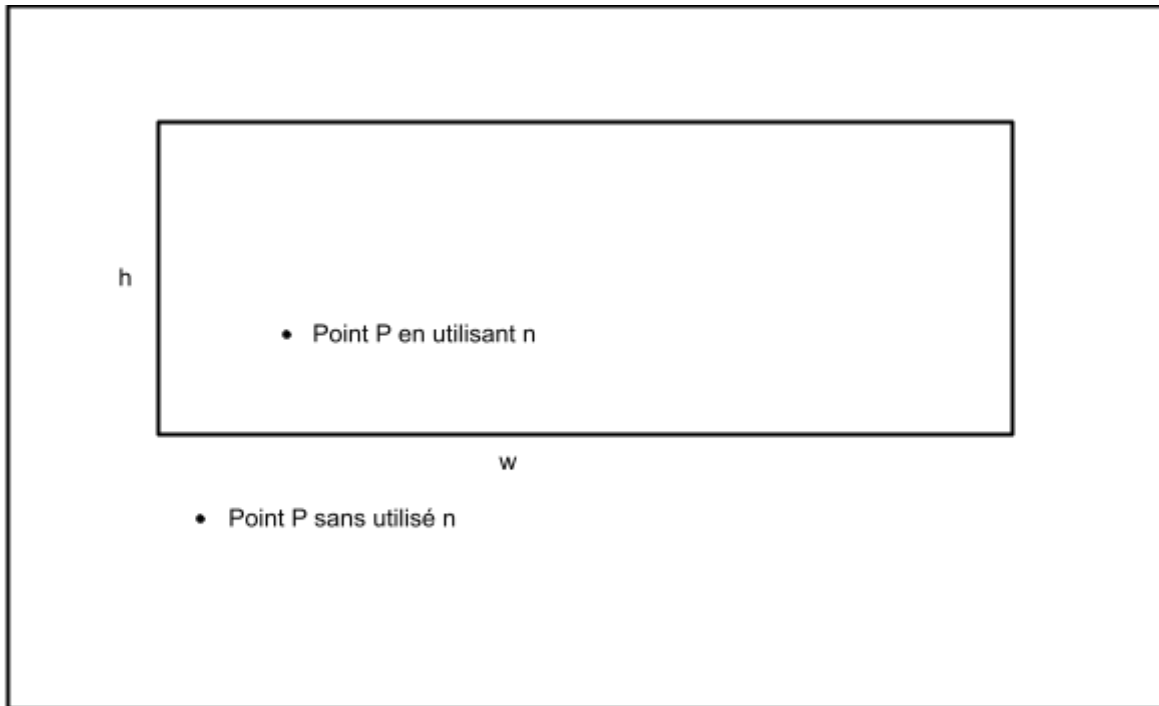
//Projection Mercator//
local_point.X = map_width * ((long_rad + Math.PI) / (2 * Math.PI));
local_point.Y = map_height / 2 - map_width / (2 * Math.PI) * Math.Log(Math.Tan(Math.PI / 4 + lat_rad / 2));

return local_point;
```

J'ai juste passé les latitudes et les longitudes en radians même si on peut aussi travailler en degré c'est l'avantage de cette projection

3. Résumé

Cette partie va résumer à l'aide d'un schéma le fonctionnement de la projection Mercator afin de mieux le visualiser



Comme on peut le voir sur le schéma n permet de calculer l'équivalent du Point P mais sur la carte que l'on souhaite utiliser

4. Source

wikipedia : https://fr.wikipedia.org/wiki/Projection_de_Mercator